

Análise da Estrutura — LotofácilPro v3

Documento gerado na etapa de análise, antes da integração com **Puter.js**.

Objetivo: mapear a arquitetura atual do sistema (ferramenta de análise estatística e fechamento reduzido 18-15-14 da Lotofácil).

1. Estrutura de Diretórios

```

LotofacilPro_v3/
├── main.py                # Orquestrador CLI (pipeline completo em 14 passos)
├── servidor.py            # Servidor Flask (API REST + serve o PWA)
├── config.py              # Configurações centrais (API, filtros, fechamento)
├── api.py                 # Download da API da Caixa com retry + cache
├──
├── Estatística / Ranking clássico
├── ├── statistics.py       # 14 análises estatísticas
├── ├── ranking.py         # Score c/ decaimento exponencial + pares/trios
├──
├── Motor Híbrido (12 critérios)
├── ├── hibrido_score.py   # Motor de pontuação híbrida (0-100)
├── ├── hibrido_pesos.py   # Gestão + otimização dos pesos
├── ├── hibrido_tendencia.py # Classificação ESQUENTANDO/ESFRIANDO/CICLO
├── ├── hibrido_perfis.py  # Perfis Conservador/Equilibrado/Agressivo
├── ├── hibrido_autoaprendizado.py # Autoaprendizado pós-concurso (ajusta pesos)
├──
├── Machine Learning
├── ├── ml_features.py     # Extração de features (sem viés futuro)
├── ├── ml_model.py        # HistGradientBoostingClassifier (sklearn)
├── ├── ml_ranking.py      # Ranking combinando ML + estatística
├── ├── ml_backtest.py     # Backtest walk-forward ML vs clássico
├──
├── Pool / Fechamento
├── ├── pool_selector.py   # Seleção inteligente do pool de 18 dezenas
├── ├── pool_adaptativo.py # Compara pools 18/19/20/21
├── ├── validator.py       # Validação matemática  $C(18,15)=816$ 
├── ├── fechamento.py     # Geração dos 24 jogos do fechamento 18-15-14
├── ├── filtros.py         # Filtros de consistência (modo AVISO)
├── ├── simulator.py       # Backtest histórico fixo e dinâmico
├──
├── AI Engine
├── ├── ai_engine.py       # "Diretor técnico": decide estratégia e rankeia jo-
├── gos
├──
├── Suporte / Saída
├── ├── acumulacao.py      # Alerta de concurso acumulado
├── ├── exports.py         # Exportação Excel/CSV/TXT/PDF
├── ├── graphs.py          # 9 gráficos PNG (matplotlib)
├── ├── utils.py           # Utilitários e compatibilidade Windows
├──
├── Interface Web (PWA)
├── ├── webapp/
├── ├── ├── index.html     # SPA completa (HTML+CSS+JS, ~1500 linhas)
├── ├── ├── manifest.json  # Manifesto PWA (instalável no celular)
├── ├── ├── sw.js          # Service Worker (cache offline)
├── ├── ├── icons/         # Ícones do app (192, 512, apple-touch)
├──
├── Runtime (criados em execução)
├── ├── cache/             # JSON por concurso + logs do AI Engine e pesos
├── ├── relatorios/        # Relatórios TXT/PDF/Excel e gráficos PNG
├── ├── jogos/             # CSVs dos jogos
├──
├── Windows launchers
├── ├── instalar.bat       # Instala Python + dependências
├── ├── rodar.bat          # Menu interativo (terminal)
├── ├── abrir_app.bat      # Inicia o servidor web (PWA)
├──
├── requirements.txt
├── README.md

```

2. Arquivo Principal de Entrada

O sistema possui **dois pontos de entrada** distintos:

Entrada	Arquivo	Uso
CLI / Terminal	<code>main.py</code>	Executa o pipeline completo em 14 passos e imprime tudo no terminal, gera exports e gráficos. Chamado por <code>rodar.bat</code> .
Web / PWA	<code>servidor.py</code>	Servidor Flask que executa o mesmo pipeline em background (thread) e expõe o resultado via API REST em JSON, consumida pela interface <code>webapp/index.html</code> . Chamado por <code>abrir_app.bat</code> .

Observação importante para a integração: não há interface desktop (Tkinter/PyQt). A UI atual é **100% web (PWA)** servida por Flask — o que torna a integração com **Puter.js** (biblioteca JavaScript client-side) natural e de baixo atrito, feita diretamente no `webapp/index.html`.

3. Módulos Python e Responsabilidades

Núcleo / Infra

- **config.py** — Diretórios, endpoint da API da Caixa (`servicebus2.caixa.gov.br/.../lotofacil`), parâmetros de retry, filtros de soma/paridade/consecutivos, matriz de índices do fechamento (24 jogos), conjuntos matemáticos (primos, fibonacci, moldura/centro). `CONCURSO_INICIO_2026 = 3577`.
- **api.py** — `ultimo_concurso()`, `buscar_concurso()`, `buscar_intervalo()`. Faz download com até 4 tentativas e cacheia cada concurso em `cache/<n>.json`.
- **utils.py** — Banner, verificação de dependências, limpeza de cache, formatação de dezenas, compatibilidade de terminal Windows.

Estatística e Ranking clássico

- **statistics.py** — 14 análises: frequência absoluta/percentual, atraso, tendência, distribuição linhas/colunas, pares/ímpares, moldura/centro, conjuntos matemáticos. Função central: `consolidar(concursos)`.
- **ranking.py** — Score com decaimento exponencial + pares/trios quentes (`calcular_pares_quentes`, `calcular_trios_quentes`, `ranking_completo`).

Motor Híbrido (12 critérios ponderados)

- **hibrido_score.py** — `calcular_scores`, `ranking_hibrido`, `tabela_ranking`.
Combina 12 critérios num score 0-100 por dezena.
- **hibrido_pesos.py** — `carregar_pesos`, `salvar_pesos`, `otimizar_pesos`.
Pesos persistidos em `cache/`; otimizados por perturbação para maximizar cobertura.
- **hibrido_tendencia.py** — `classificar_todas` → ESQUENTANDO / ESFRIANDO / CICLO_RETORNO / NEUTRO, com setas e pressão.
- **hibrido_perfis.py** — Perfis Conservador / Equilibrado / Agressivo
(`top18_por_perfil`, `PERFIS_VALIDOS`).
- **hibrido_autoaprendizado.py** — `executar_ciclo_autoaprendizado`: avalia o ciclo anterior e reotimiza os pesos após cada concurso.

Machine Learning

- **ml_features.py** — `extrair_features`: vetor por (concurso, dezena) usando SÓ dados anteriores (sem viés futuro).
- **ml_model.py** — `treinar`, `prever_probabilidades` com `HistGradientBoostingClassifier` (sklearn).
- **ml_ranking.py** — `ranking_ml`, `top18_ml`, `comparar_rankings`
(média ponderada ML + estatística).
- **ml_backtest.py** — `backtest_comparativo` walk-forward ML vs clássico.

Pool e Fechamento

- **pool_selector.py** — `selecionar_pool`, `validar_pool` (checklist de qualidade 0-100), `relatorio_selecao`.
- **pool_adaptativo.py** — `analisar_pools`, `melhor_pool` (18/19/20/21).
- **validator.py** — `validar_matriz`: valida exaustivamente as $C(18,15)=816$ combinações (garantia matemática do fechamento).
- **fechamento.py** — `gerar_jogos` (24 jogos a partir de 18 dezenas), `custo_fechamento`.
- **filtros.py** — `filtrar_jogos` em **modo aviso** (alertas, não exclusões).
- **simulator.py** — `backtest` (fixo) e `backtest_dinamico` (sem viés futuro).

AI Engine (camada de decisão)

- **ai_engine.py** — Ponto de entrada `analisar(...)` → `AnaliseCompleta`.
Responsabilidades:
 1. `_avaliar_estado_mercado` — classifica o momento (AQUECIDO/FRIO/CÍCLICO/...).
 2. `_escolher_estrategia` — decide perfil (conservador/equilibrado/agressivo) com base em backtests, tendências e histórico próprio de decisões.
 3. `_rankear_jogos` — dá score de confiança 0-100 a cada um dos 24 jogos e classifica em ALTA/MÉDIA/BAIXA.
 4. `_gerar_explicacao` — texto explicativo em linguagem natural.
 5. Persiste histórico de decisões em `cache/ai_engine_log.json`.
- **Importante:** as explicações do AI Engine são hoje **geradas por regras/templates locais** (heurísticas em Python), não por um LLM. Este é o principal ponto onde o Puter.js (LLM client-side) pode agregar valor.

Saída

- **exports.py** — Excel (6 abas), CSV (ranking/jogos/histórico/pares), TXT, PDF.

- **graphs.py** — 9 gráficos PNG via matplotlib.
- **acumulacao.py** — Alerta de concurso acumulado.

4. Fluxo de Execução Atual

Ambos os pontos de entrada seguem o **mesmo pipeline** de 14 passos + AI Engine:

1. Acumulação (alerta)	→	acumulacao.py
2. Download concursos (Caixa)	→	api.py (cache local)
3. Estatísticas	→	statistics.consolidar
4. Pares e trios quentes	→	ranking.py
5. Tendências	→	hibrido_tendencia.classificar_todas
6. Scores híbridos (12 crit.)	→	hibrido_score.ranking_hibrido
7. Autoaprendizado dos pesos	→	hibrido_autoaprendizado
8. Modelo ML	→	ml_model.treinar / prever
9. Seleção do pool (18)	→	pool_selector.selecionar_pool
10. Validação do pool	→	pool_selector.validar_pool
11. Fechamento matemático	→	validator + fechamento (24 jogos)
12. Filtros (modo aviso)	→	filtros.filtrar_jogos
13. Backtests (fixo + dinâmico)	→	simulator
[AI ENGINE] estado, estratégia, rankeamento, explicação → ai_engine.analisar		
14. Exportar (CLI) / montar JSON (Web)		

- **CLI (main.py)**: imprime cada passo no terminal, gera gráficos e todos os exports, e mostra um resumo final.
- **Web (servidor.py)**: `_rodar_analise()` roda o pipeline numa thread, atualizando `_estado` (status/progresso/etapa). O frontend faz polling em `/api/status` e, ao concluir, busca `/api/resultado` com o payload JSON completo.

5. Interface do Usuário

Tecnologia: PWA (Progressive Web App) — HTML + CSS + JavaScript puro (sem framework), servida pelo Flask. **Não há Tkinter/PyQt.**

- `webapp/index.html` (~1500 linhas) — SPA com CSS e JS embutidos, tema escuro, layout mobile-first (instalável no celular via manifest + service worker).
- **Fluxo do frontend**:
 - Botão → `POST /api/analisar` (dispara o pipeline em background).
 - `pollStatus()` → `GET /api/status` a cada 700 ms (barra de progresso).
 - `carregarResultado()` → `GET /api/resultado` → `renderizarTudo(dados)`.
- **Abas (tabbar)**: Resumo, Jogos, Ranking, Backtest, Híbrido, **AI**, (ML).
- **Aba AI** (`#tabAI` , render em `renderAI()` linha ~1434): mostra estratégia, confiança, distribuição de jogos por confiança, **alertas**, um bloco de **“Análise / Explicação das decisões”** (`#aiExplicacao`) e a lista de jogos rankeados. É a aba mais natural para exibir texto gerado por LLM via Puter.js.

Endpoints Flask existentes (`servidor.py`):

Método	Rota	Função
-----	-----	-----

GET	/	Serve webapp/index.html
POST	/api/analisar	Dispara o pipeline (background thread)
GET	/api/status	status/progresso/etapa/erro
GET	/api/resultado	payload JSON completo da análise
POST	/api/cache/limpar	limpa o cache local

6. Dependências (requirements.txt)

```
requests>=2.31.0      # download da API da Caixa
openpyxl>=3.1.0      # exportação Excel
matplotlib>=3.8.0    # gráficos PNG
reportlab>=4.0.0      # exportação PDF
flask>=3.0.0          # servidor web / API REST
scikit-learn>=1.3.0  # modelo ML (HistGradientBoosting)
numpy>=1.24.0         # cálculo numérico
xgboost>=2.0.0        # listado (o ml_model usa sklearn como equivalente)
scipy>=1.11.0         # suporte científico
```

- **Frontend não possui dependências JS** hoje (nenhum `<script src>` externo, nenhum CDN). Todo o JS é inline em `index.html`. **Não há Puter.js ainda.**

7. Pontos de Integração para o Puter.js

O Puter.js é uma biblioteca **client-side** (`<script src="https://js.puter.com/v2/">`) que oferece acesso a LLMs (e outros serviços) direto do navegador, sem backend nem chaves de API. Pontos de integração recomendados, do mais direto ao mais profundo:

A. Frontend (webapp/index.html) — caminho principal

1. **Incluir o script do Puter.js** no `<head>` do `index.html` (`<script src="https://js.puter.com/v2/"></script>`).
2. **Enriquecer a aba AI** (`renderAI` , `#aiExplicacao`): enviar o objeto `d.ai` + contexto (`estado_mercado` , `estrategia` , `jogos_rankeados` , `backtest` , `tendências`) para `puter.ai.chat(...)` e exibir uma análise em linguagem natural gerada por LLM, complementando/ substituindo o texto por template atual.
3. **Chat/assistente interativo**: adicionar um campo onde o usuário faz perguntas (“por que o jogo 7 tem alta confiança?”) e o Puter.js responde usando o payload JSON já disponível em `dadosAtuais` .

B. Dados já prontos para alimentar o LLM

O endpoint `/api/resultado` já entrega um JSON rico e estruturado (`ranking` , `dezenas_18` , `pares_quentes` , `backtest` , `hibrido` , `ml` , `pool_inteligente` , `ai.jogos_rankeados` , `ai.explicacao` , `ai.alertas`). Esse payload é o **contexto ideal** para prompts do Puter.js — não é preciso recalculá-lo no cliente.

C. Backend (`ai_engine.py`) — opcional / híbrido

- Hoje `_gerar_explicacao` e as explicações por jogo são **baseadas em regras**.
- Como Puter.js é client-side, a estratégia mais limpa é manter o `ai_engine.py` produzindo os **dados/decisões numéricas** e delegar a **redação em linguagem natural** ao Puter.js no frontend. Alternativamente, criar um novo endpoint Flask (ex.: `/api/ia/explicar`) só se for necessário orquestrar do lado servidor.

D. Considerações

- **Offline/PWA:** o `sw.js` faz cache; chamadas ao Puter.js exigem internet e devem degradar graciosamente para o texto por template quando offline.
- **Sem backend/API key:** Puter.js usa o modelo “User Pays”, eliminando necessidade de gerenciar chaves — bom para uma app distribuída como PWA.
- **Ponto de menor atrito:** aba **AI** do `index.html` + payload de `/api/resultado` . É onde a integração deve começar.

Resumo executivo

- App **web (PWA) + Flask**, sem GUI desktop.
- Pipeline maduro de 14 passos, com estatística, motor híbrido, ML e um **AI Engine baseado em regras** que decide estratégia e rankeia os 24 jogos.
- `ai_engine.py` **já existe** e produz decisões + explicações por template.
- A integração com **Puter.js** deve ocorrer no **frontend** (`webapp/index.html` , aba AI), consumindo o JSON já disponível em `/api/resultado` para gerar análises em linguagem natural e um assistente interativo.